Dyad Backend Server — Architecture Report

Generated: June 12, 2026
Application: Dyad Practice Solutions Backend API
Stack: Node.js · Express 5 · PostgreSQL · Google Calendar · Calendly · Gmail

---

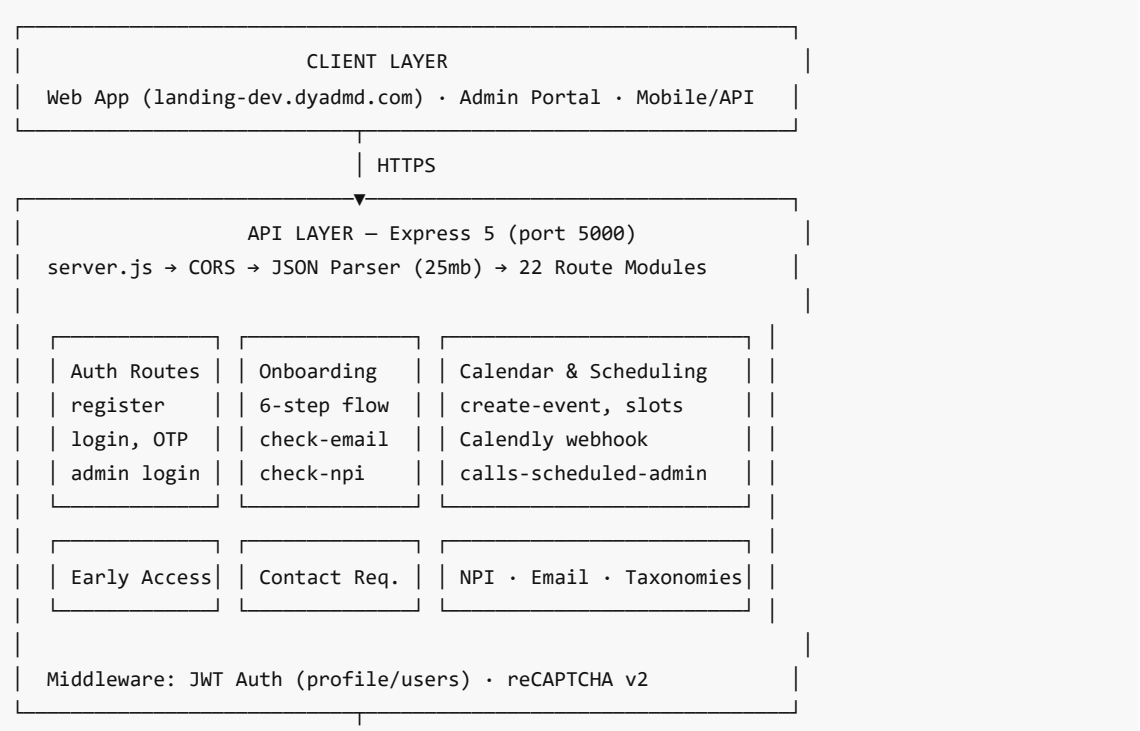
1. Executive Summary

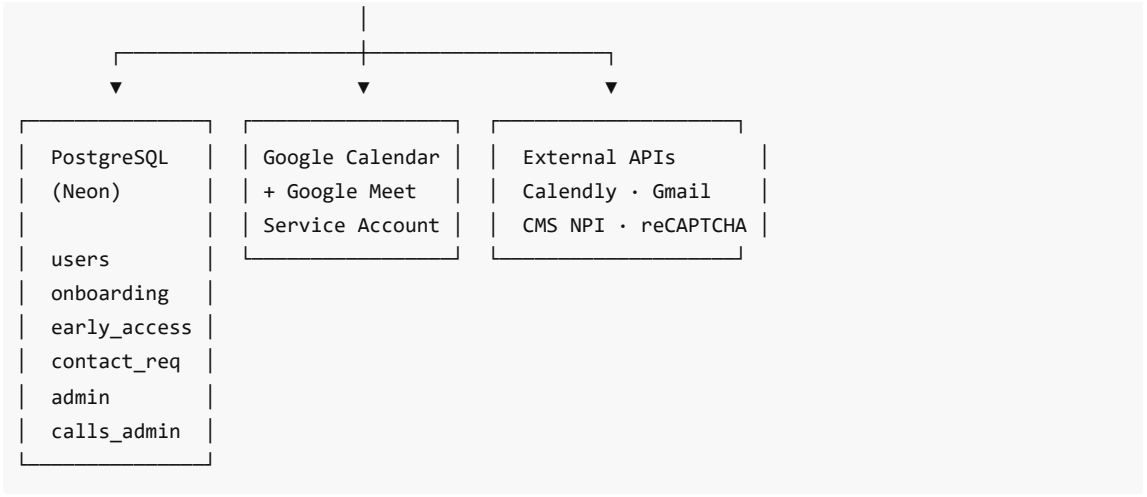
The Dyad backend is an **Express 5 monolith** that powers healthcare provider onboarding, early-access lead management, contact requests, calendar scheduling, and admin workflows. It connects to **PostgreSQL** (Neon), **Google Calendar/Meet**, **Calendly**, **Gmail (Nodemailer)**, and the **CMS NPI Registry**.

Dimension	Current State	Large-Scale Ready?
Architecture	Express monolith, 22 route modules	Partial — needs decomposition
Database	PostgreSQL (Neon)	Yes — with pooling + replicas
Auth	JWT on 3 endpoints only	No — major gap
Scheduling	Google Calendar + Calendly	Yes — externalized
Email	Inline Gmail/Nodemailer	No — needs queue
Healthcare data	NPI, onboarding PII stored	Needs compliance hardening
Observability	Console logs only	No — needs APM/monitoring

---

2. High-Level System Architecture





---

3. Application Layer Structure

```
server.js (entry point)
├── config/db.js          → PostgreSQL Pool (DATABASE_URL + SSL)
├── middleware/
│   ├── auth.js           → JWT Bearer verification
│   └── recaptcha.js      → Google reCAPTCHA siteverify
├── utils/
│   ├── generateOtp.js    → OTP generation
│   └── meetingLink.js    → Google Meet link helpers
├── routes/ (22 modules)
│   ├── Authentication: register, login, adminLogin, refresh
│   ├── Users: profile, users, verifyEmail, verifyOtp
│   ├── Password: sendEmailOtp, forgotPassword
│   ├── CRM: contactRequests, earlyAccess, callsScheduledAdmin
│   ├── Onboarding: onboardingSteps, onboardingClient
│   ├── Scheduling: calendar, calendly-webhook
│   ├── Email: sendEmail, onboardingScheduleConfirmation
│   └── Misc: npiRegistry, taxonomies, recaptcha, apiDocumentation
└── database/             → SQL schemas + migrations
```

---

4. API Route Groups (~55+ endpoints)

Authentication & Users

Endpoint	Method	Purpose
/api/register	POST	User registration (bcrypt hash)
/api/login	POST	Login → JWT access + refresh tokens
/api/admin/login	POST	Admin table login
/api/refresh	POST	Refresh access token
/api/profile	GET	Authenticated user profile

/api/users	GET/PUT	User management (partial auth)
/api/send-email-otp	POST	Email OTP verification
/api/verify-otp	POST	Verify OTP
/api/forgot-password	POST	Password reset flow

Onboarding (6-step flow)

Endpoint	Method	Purpose
/api/onboarding/check-email	POST	Check if email registered
/api/onboarding/step/:step	POST	Save steps 1–6 (JSONB)
/api/onboarding	GET	List all onboarding records
/api/onboarding/:id	GET	Get onboarding by ID
/api/onboarding-client/check-npi	POST	Check NPI availability

Calendar & Scheduling

Endpoint	Method	Purpose
/api/create-event	POST	Google Calendar event + Meet link
/api/update-event	POST/PATCH/PUT	Update/reschedule event
/api/calendar/available-slots	GET	Available time slots
/api/calls-scheduled-admin	POST/GET	Admin scheduled calls
/api/send-onboarding-schedule-confirmation	POST	Confirmation email

Early Access & CRM

Endpoint	Method	Purpose
/api/api-early-access	POST/GET	Early access applications
/api/contact-requests	POST/GET/PATCH	Contact form CRUD
/api/npi/registry	POST	CMS NPI Registry proxy

---

5. Database Schema

Table	Purpose	Key Columns
users	User accounts	email, password_hash, npi, email_verified
admin	Admin portal login	username, password_hash
contact_requests	Landing page leads	name, email, status, scheduled_time

onboarding_steps	6-step onboarding	onboarding_id, step_N_payload, meeting_id
early_access_requests	Beta program signups	email, npi, practice_name, status
calls_scheduled_admin	Admin call records	email, contact_name, scheduled_at, meeting_id
provider_taxonomies	Medical taxonomy codes	code, description

---

6. Core Business Flows

Onboarding + Scheduling

1. User checks email → POST /onboarding/check-email
2. User completes steps 1–6 → POST /onboarding/step/:step
3. User schedules call → POST /create-event (Google Calendar + Meet)
4. Meeting synced → onboarding_steps.meeting_id, call_event_id
5. Admin record saved → POST /calls-scheduled-admin
6. Confirmation email → POST /send-onboarding-schedule-confirmation

Authentication

1. Register → POST /register
2. Verify email → POST /send-email-otp → POST /verify-otp
3. Login → POST /login (access token 15min, refresh 7 days)
4. Protected routes → Bearer JWT on /profile, /users

---

7. External Integrations

Service	Usage	Auth Method
PostgreSQL (Neon)	Primary datastore	DATABASE_URL + SSL
Google Calendar	Event creation, slots	Service account JSON
Google Meet	Dynamic meeting links	Calendar conferenceData
Calendly	Legacy slots/booking	CALENDLY_TOKEN
Gmail (Nodemailer)	All transactional email	EMAIL_USER / EMAIL_PASS
CMS NPI Registry	Provider validation	Public REST API
Google reCAPTCHA v2	Bot protection	RECAPTCHA_SECRET_KEY_V2

---

8. Deployment Architecture

Target	Config
Development	nodemon server.js → localhost:5000
Heroku	Procfile: web: npm start
Docker	Node 20 Alpine + docker-compose

Database	Neon PostgreSQL (production) / Postgres 15 (local)

Key Environment Variables: DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, EMAIL_USER, EMAIL_PASS, CALENDLY_TOKEN, MEETING_LINK, RECAPTCHA_SECRET_KEY_V2

---

9. Scalability — Pros & Cons

Pros

- **Stateless API (mostly)** — JWT auth, no server-side sessions for most routes
- **PostgreSQL** — Solid relational DB; Neon supports connection pooling
- **Modular routes** — 22 files by domain; easier to split into microservices later
- **JSONB onboarding** — Flexible step storage without schema migrations
- **Managed external services** — Calendar, Calendly, NPI offload heavy work
- **Indexed queries** — email, NPI, meeting_id, scheduled_at indexed
- **Docker-ready** — Supports horizontal scaling behind load balancer

Cons

- **Monolithic architecture** — Single Node process; one slow route affects all
- **In-memory OTP stores** — Breaks with multiple instances or restarts
- **No caching layer** — No Redis for sessions, rate limits, or hot reads
- **No job queue** — Email and calendar calls run inline in HTTP requests
- **Runtime DDL on startup** — Table creation at boot risky with many replicas
- **No connection pool tuning** — Default pg Pool may exhaust DB connections
- **Dual scheduling stack** — Google Calendar + Calendly adds complexity
- **No automated tests** — Scaling changes are risky without coverage

Verdict: Suitable for **hundreds to low thousands** of users. For **tens of thousands+**, needs Redis, job queue, read replicas, rate limiting, and background workers.

---

10. Security — Pros & Cons

Pros

- **Password hashing** — bcrypt/bcryptjs for user and admin passwords
- **JWT access + refresh** — Short-lived access (15 min) + refresh (7 days)
- **Email verification** — OTP and token flows before full access
- **reCAPTCHA support** — Google reCAPTCHA v2 middleware available
- **Parameterized SQL** — $1, $2 placeholders prevent SQL injection
- **SSL to PostgreSQL** — DATABASE_URL uses sslmode=require
- **Separate admin table** — Admin credentials isolated from users
- **Service account gitignored** — Google credentials not in repo

Cons

- **Most endpoints unauthenticated** — Onboarding, early access, admin routes public
- **Admin JWT not enforced** — Admin login issues token but routes don't verify it
- **GET /userslist is public** — Exposes full user list without auth
- **CORS allows all origins** — Fallback bypasses whitelist in practice
- **In-memory OTP** — Lost on restart; not multi-instance safe
- **No rate limiting** — Brute-force login and API abuse possible
- **reCAPTCHA not wired globally** — Only standalone endpoint

- **No RBAC middleware** — Role field exists but not enforced
- **PII in logs** — Calendar routes log request details to console
- **Calendly webhook unverified** — No signature verification
- **Healthcare PII** — NPI, emails, practice data need HIPAA-aligned controls

Verdict: Fine for **MVP / early access beta**. **Not production-hardened** for large customer base with healthcare PII without auth middleware, rate limiting, Redis OTP, and audit logging.

---

11. Recommended Evolution Path

```
Phase 1 (Short term)
  → Protect admin/PII routes with JWT + role middleware
  → Add rate limiting
  → Move OTP to Redis

Phase 2 (Medium term)
  → Background jobs for email/calendar (BullMQ/SQS)
  → Structured logging + health check endpoint
  → API integration tests

Phase 3 (Long term)
  → Read replicas + caching
  → Event-driven webhooks
  → Compliance audit trail for healthcare data
  → Optional microservices split
```

---

12. Overall Assessment

This application is a **feature-rich healthcare onboarding and lead-management backend** with strong domain coverage:

- NPI validation via CMS Registry
- Multi-step onboarding (6 steps, JSONB payloads)
- Google Calendar scheduling with dynamic Meet links
- Early access program management
- Admin scheduled call tracking
- Transactional email via Gmail

It is well-structured for a **growing startup phase**, but requires **authorization hardening, distributed state (Redis), and async processing** before serving a large, security-sensitive customer base at scale.

Report generated for Dyad Practice Solutions Backend Server